

Presented by Team-08

DRLE: DECENTRALIZED REINFORCEMENT LEARNING AT THE EDGE FOR TRAFFIC LIGHT CONTROL IN THE IOV

1. UPDATES FROM PHASE 1

- Finalized system architecture (Edge  RL Controller)
- Defined traffic topology (5×5 grid of intersections)
- Implemented iFogSim2 simulation with:
 - Edge hierarchy (ES1 → Intersections)
 - Sensor/actuator logic at each intersection
- Completed two baseline control policies:
 - Fixed Timer Control
 - Actuated Threshold-Based Control
- Integrated JSON-based communication plan for RL service (DRLE-style)
- Set up metric logging (SLA miss %, latency, network usage, halting %, etc.)

2. ARCHITECTURE DESIGN

- **The system follows a edge computing architecture:**

Vehicles (data sources)

Intersections (IoT nodes with sensors + actuators)

Edge Server Tier 1 (ES1) – real-time control

- **Each intersection sends telemetry (e.g., queue length, avg speed) to its ES1 node every 1 second.**
- **Control logic at ES1 supports 3 policies:**
 - Timer (fixed cycle)**
 - Actuated (rule-based)**
 - DRLE (decentralized RL via external Python API – to be implemented)**

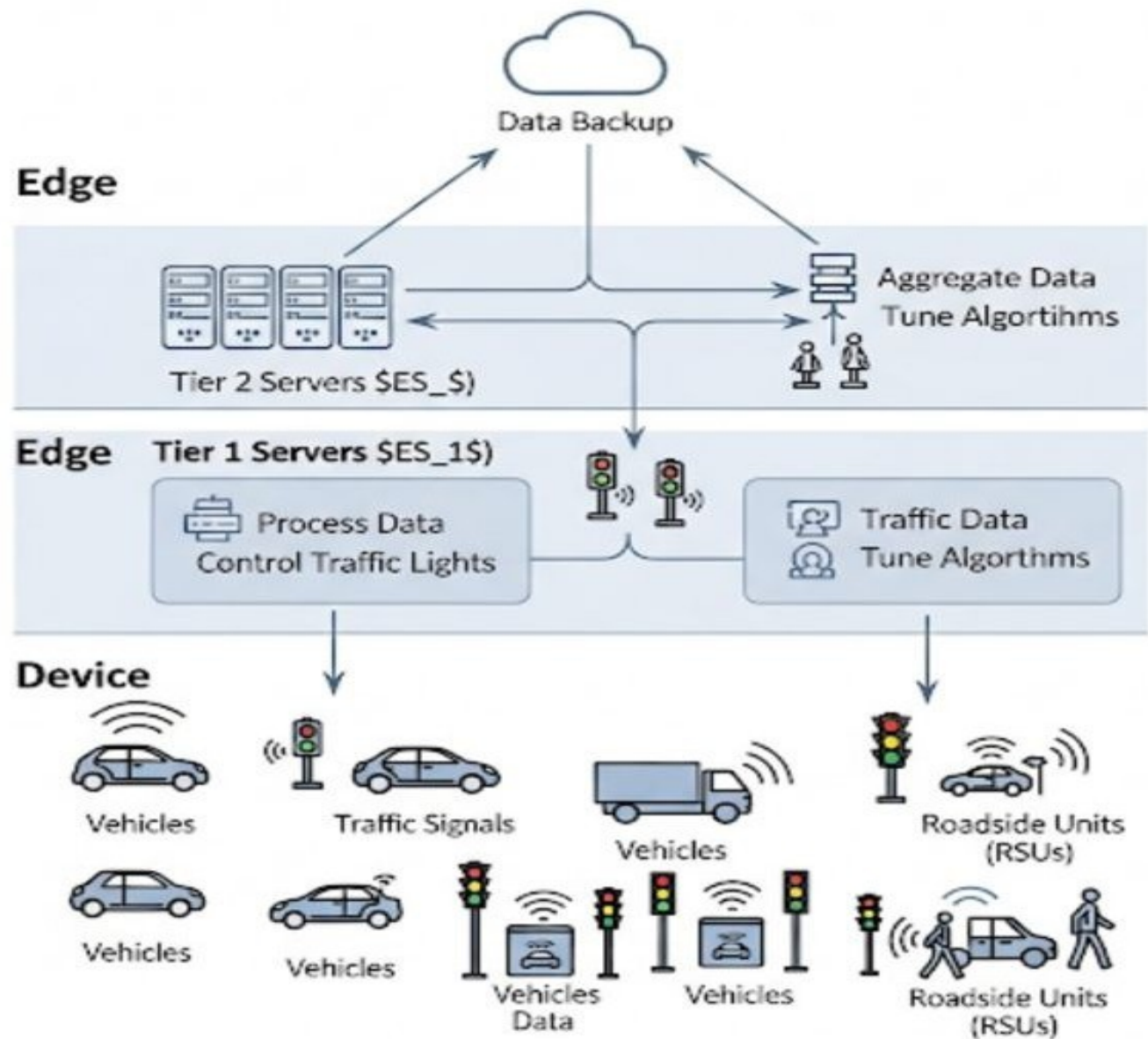
In DRLE mode:

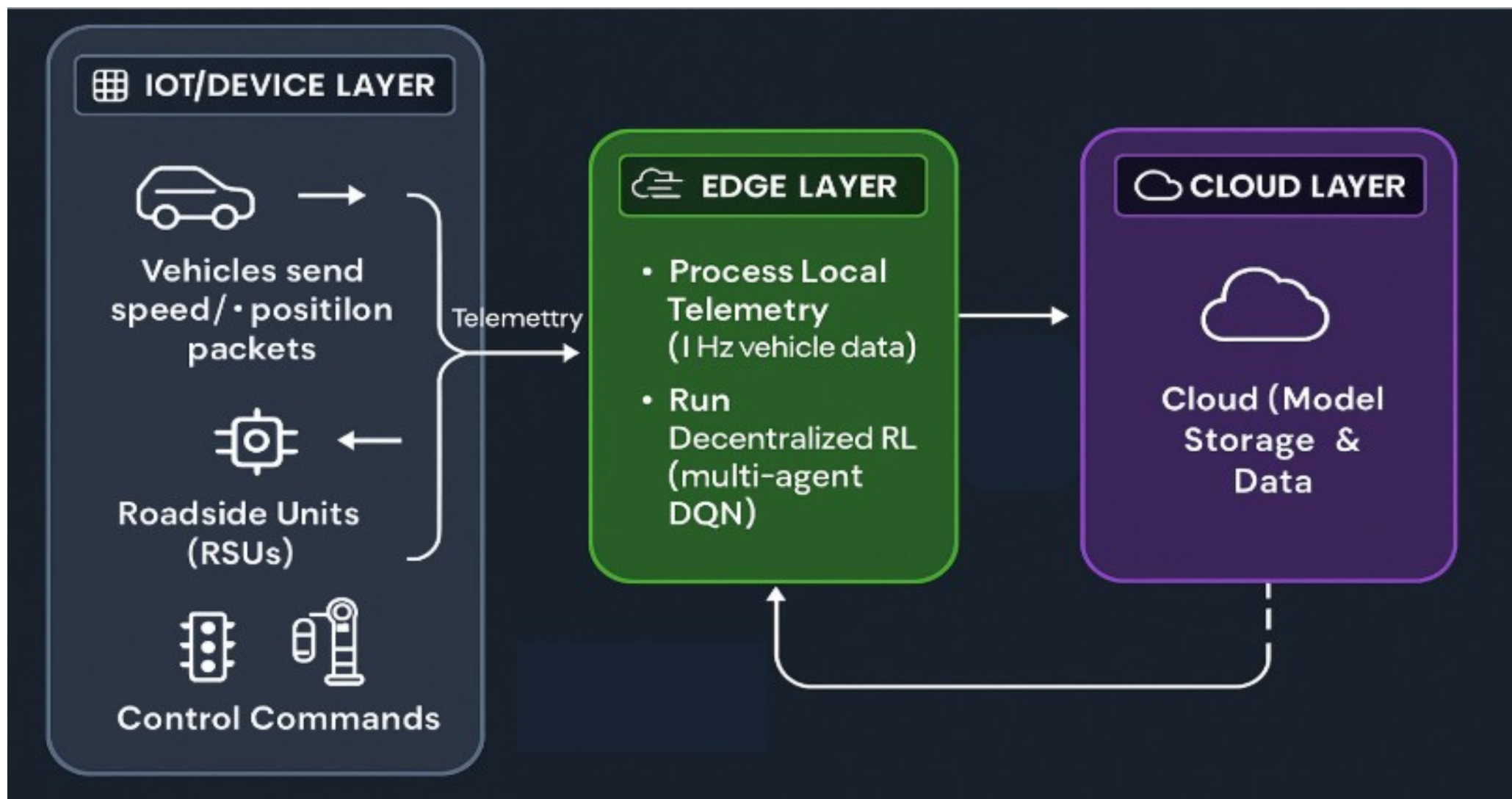
- **ES1 collects intersection states and calls a DQN model to decide actions.**
- **Control decisions are applied to traffic lights within the 1-second loop.**

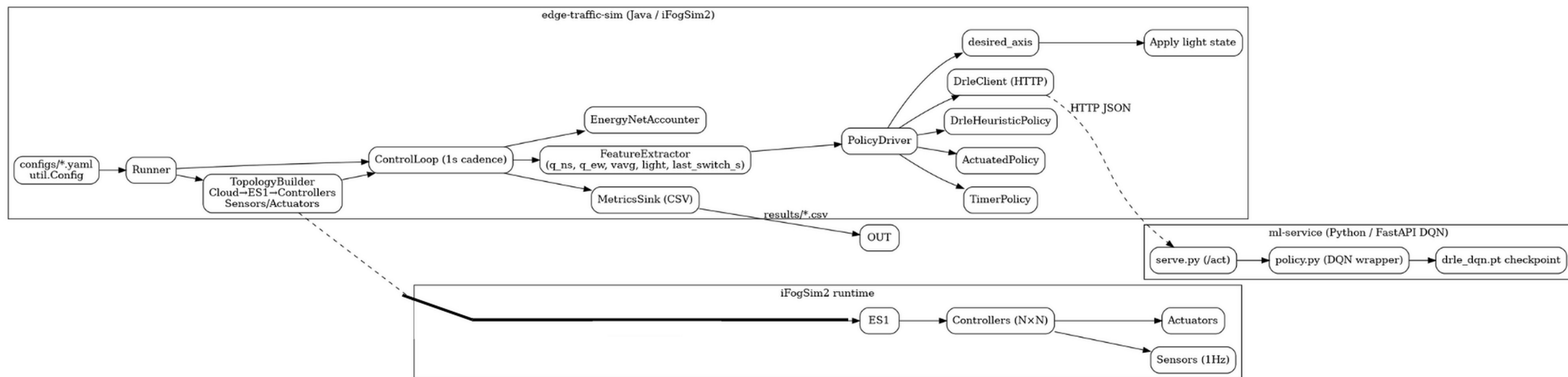
ES1 nodes also log metrics:

- **Control latency, SLA compliance, uplink/downlink usage**
- **Energy usage(toy model) and traffic performance (queue proxy, halting %)**
- **This modular, scalable design ensures fast, localized decisions with optional RL integration.**

Client Architecture







STRATEGIES & TOOLS USED

Tools:

- **iFogSim2 (Java) – Edge computing simulator**
- **Java for simulation logic**
- **Python (FastAPI planned) – RL model API**
- **Git – version control**

Strategies:

- **Policy abstraction: Timer, Actuated, and DRLE policies are plug-and-play**
- **Decentralized edge execution: each ES1 makes decisions independently (scalable)**
- **Metric logging (CSV) for analysis**
- **Staged implementation:**
 - Timer & Actuated first**
 - DRLE integration next (via CI team)**

EXPLORER

CAPSTONE-PROJECT-08_LAZYS...

.vscode

CI-traffic-sim

Documentation

edge-traffic-sim

configs

base-5x5.yaml

results

compare-base-5x5

avg_halting_pct.png

avg_latency_ms.png

avg_queue.png

es1_power.png

summary.csv

run-base-5x5-actuated-5x5.csv

run-base-5x5-timer-5x5.csv

src\main\java\com\team\traffic

features

FeatureExtractor.java

metrics

EnergyNetAccounter.java

PolicyDriver.java

summary.csv

edge-traffic-sim > results > compare-base-5x5 > summary.csv

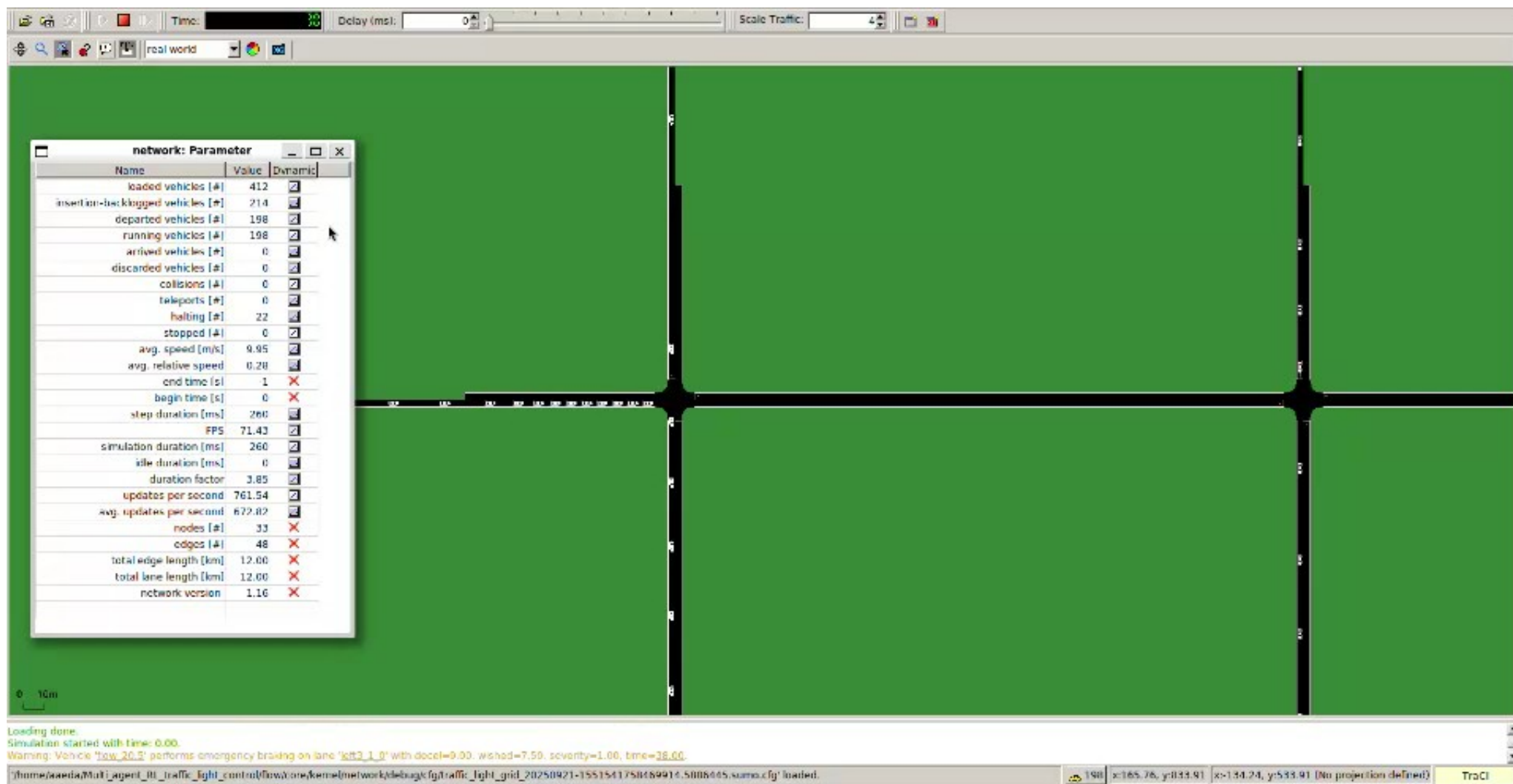
1 policy,avg_queue,avg_queue_ns,avg_queue_ew,avg_halting_pct,avg_latency_ms,total_bytes_up_MB,total_bytes_down_MB,avg_es1_power_w,sla

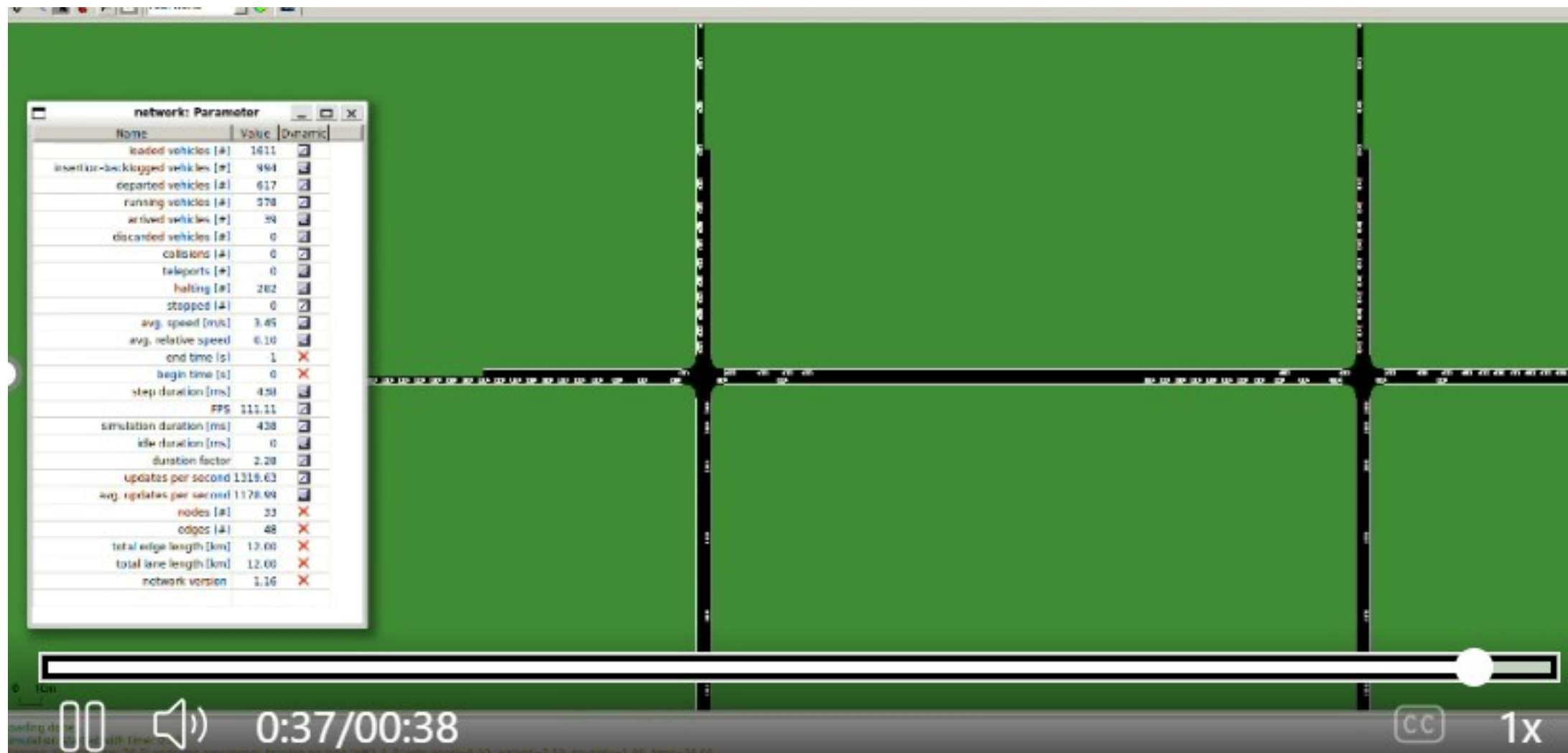
2 TIMER,631.1239,552.8000,709.4478,0.5012,226.0000,163.0783,17.1661,45.2500,0.0000,3600.0000

3 ACTUATED,630.4806,778.5299,482.4313,0.8204,226.0000,163.0783,17.1661,45.2500,0.0000,3600.0000

4 Δ% vs TIMER (positive = improvement),0.1019,-40.8339,31.9991,-63.6806,0.0000,0.0000,0.0000,0.0000,,

5





Role of Computational Intelligence (CI) in Traffic Light Control

- Computational Intelligence (CI) is used to optimize **urban traffic light control**.
- Traditional methods:
 - **Static control** → pre-timed signals, not adaptive.
 - **Actuated control** → responds to sensor triggers, but still rule-based.
- CI introduces **machine learning-based adaptive control**.
- Our CI contribution in this project:
 - Implement **Non-RL baselines** (Static, Actuated).
 - Implement **RL-based agents** (Single-Agent PPO for Figure-Eight and Traffic Light Grid).
- CI enables:
 - **Decentralized control** (each agent controls its intersection).
 - **Learning from data** instead of fixed rules.
 - **Scalability** → multiple agents across city-scale networks.

```
if alg_run == "PPO":
    agent_cls = get_agent_class(alg_run)
    config = deepcopy(agent_cls._default_config)
    config["gamma"] = 0.999 # discount rate
    config["model"].update({"fcnet_hiddens": [32, 32,
32]})
    config["use_gae"] = True
    config["lambda"] = 0.97
    config["kl_target"] = 0.02
    config["num_sgd_iter"] = 10

elif alg_run == "DQN":
    os.environ["FLOW_ALGO"] = flags.algorithm
    agent_cls = get_agent_class(alg_run)
    config = deepcopy(agent_cls._default_config)
    config["clip_actions"] = False
    config["timesteps_per_iteration"] = horizon * n_rollouts
    config["hiddens"] = [512]
    config["lr"] = 0.0000625
    config["schedule_max_timesteps"] = 2000000
    config["buffer_size"] = 1000000
    config["target_network_update_freq"] = 8000
```

Implemented Models & Reward Mechanisms

RL Models Implemented in CI side:

- **Single-Agent PPO on Figure-Eight network**
(tests vehicle coordination at a loop with one intersection).
- **Single-Agent PPO on Traffic Light Grid**
(tests coordination across a small grid of intersections).
- **Non-RL baselines** included for fair comparison.
- **Reward Calculation:**(per timestep)

$$R = w_1 \cdot R_{\text{speed}} + w_2 \cdot R_{\text{halting}} + w_3 \cdot R_{\text{queue}}$$

Speed reward:

$$R_{\text{speed}} = \frac{1}{N} \sum_{i=1}^N \frac{v_i}{v_{\text{desired}}}$$

Halting penalty:

$$R_{\text{halting}} = - \text{num_halting_vehicles}$$

Queue length penalty:

$$R_{\text{queue}} = - \frac{1}{N} \sum_{i=1}^N q_i$$

• **RL Loop:**

- **State:** traffic conditions (speeds, queues, light states).
- **Action:** traffic light phase decisions.
- **Reward:** immediate traffic improvement (reduced queue, higher avg. speed).
- **Policy Update:** PPO agent updates weights based on experience.

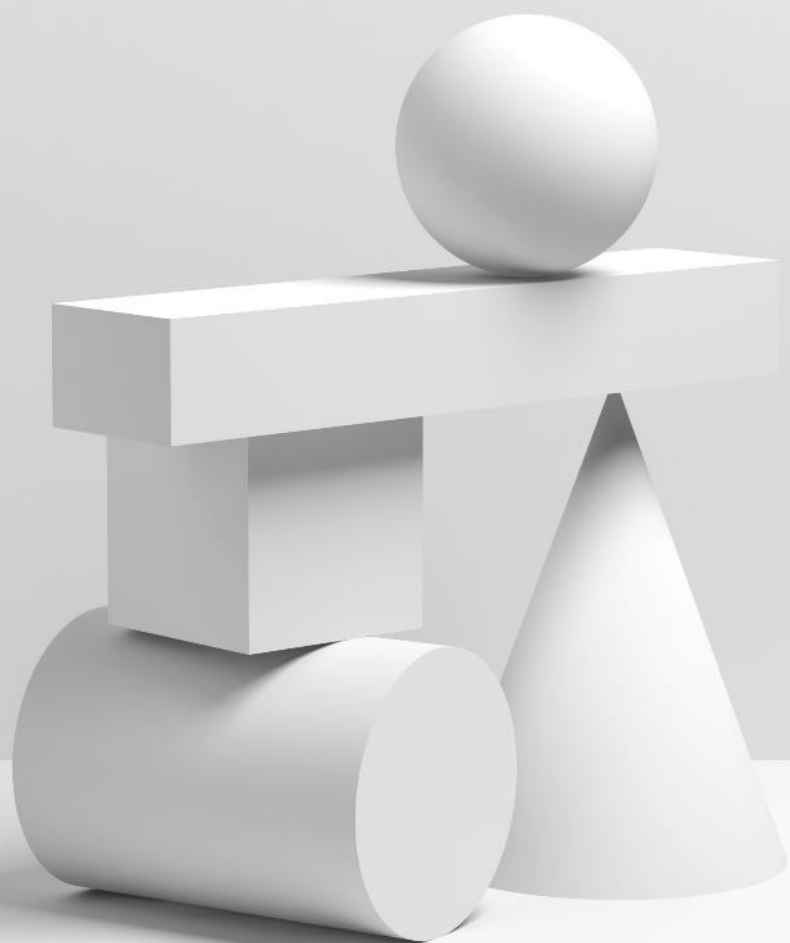
CI Algorithm Performance (Evaluation from Paper & Our Models)

- PPO achieves **significant reduction in queue time** compared to Non-RL.
- Halting vehicles also reduced, though still higher than DQN in paper.
- Non-RL actuated is better than static, but cannot adapt like PPO.
- CI-based methods show clear advantage in **queue reduction** and **speed stabilization**.
- Our CI work focused on **implementing, training, and evaluating these PPO and baseline models**.

Algorithm	Halting Vehicles	Queue Time (s)	Queue Length (m)	Speed (m/s)
Non-RL: Static	95 ± 40	14.23 ± 10.25	19.82 ± 12.17	13.39 ± 3.20
Non-RL: Actuated	13.14 ± 6.63	3.58 ± 4.65	14.85 ± 10.57	18.25 ± 1.96
Single-Agent PPO	9.78 ± 3.51	0.76 ± 1.26	8.20 ± 3.38	15.06 ± 1.52
Single-Agent DQN	2.06 ± 1.78	0.25 ± 0.50	7.31 ± 1.29	16.67 ± 1.56

Future Improvements:

- **Multi-Agent RL (Decentralized DRLE)** for city-scale scalability
- **Hybrid models** (rule-based + RL for safety-critical intersections)
- **Integration with real-world traffic data** (API / IoV sensors)
- **Cloud & Edge Deployment** for faster training & inference



Q & A

Panel discussion session